

🌟 DAY - 23: JavaScript Promises - From Beginner to PRO 🌟

Created by: Neeraj | [LinkedIn: neeraj-kumar1904](#) 📁 | [X: @_19_neeraj](#) 🐦 | [GitHub: Neeraj05042001](#) 🧑

📖 1. Introduction to JavaScript Promises

JavaScript Promises are a powerful feature that helps us handle asynchronous operations in a cleaner and more manageable way compared to traditional callbacks. A Promise represents the eventual completion or failure of an asynchronous operation and its resulting value.

😬 Why Do We Need Promises?

- 🔥 **Callback Hell:** Traditional callbacks can lead to deeply nested code that's hard to read and maintain
- ⚡ **Better Error Handling:** Promises provide a unified way to handle errors
- 🧩 **Composition:** Promises can be easily chained and combined
- 📄 **Readability:** Promise-based code is more readable and follows a linear flow

🎯 2. What is a Promise in JavaScript?

A Promise is an object that represents the eventual completion or failure of an asynchronous operation. It's like a placeholder for a value that will be available in the future.

🍕 Real-World Analogy

Think of a Promise like ordering food at a restaurant:

- 📄 You place an order (create a promise)
- 📄 You get a receipt (the promise object)
- 🧑 The kitchen prepares your food (asynchronous operation)
- ✅ Either you get your food (resolved) or there's an issue (rejected)

★ Promise Characteristics

- 🔒 Promises are immutable once settled
- 1 They can only be resolved or rejected once

- 🗑️ They provide a clean way to handle async operations

🗨️ Knowledge Check:

- ▶️ 💡 What is the main advantage of Promises over callbacks?

🔧 3. How to Create Promises

📄 Basic Promise Syntax

```
let promise = new Promise(function(resolve, reject) {  
  // 🗑️ Executor function  
  // Your asynchronous logic goes here  
});
```

⚙️ The Executor Function

The executor function is called immediately when the Promise is created. It receives two parameters:

- ✅ **resolve**: Function to call when operation succeeds
- ❌ **reject**: Function to call when operation fails

🎨 Example: Creating Simple Promises

```
// 🟢 Promise that resolves  
let promise1 = new Promise(function(resolve, reject) {  
  resolve("Hey, I am done!");  
});  
  
// 🔴 Promise that rejects  
let promise2 = new Promise(function(resolve, reject) {  
  reject("Something is not right!");  
});
```

🗨️ Knowledge Check:

- ▶️ 😬 What happens if you call both `resolve()` and `reject()` in the same executor?

📊 4. Promise States and Values

🔄 Promise States

Every Promise has one of three states:

1. 🕒 **Pending**: Initial state, neither fulfilled nor rejected
2. ✅ **Fulfilled**: Operation completed successfully
3. ❌ **Rejected**: Operation failed

📋 Promise Results

Along with states, Promises have results:

1. 🙋 **undefined**: Initially when state is pending
2. 💎 **value**: When resolve(value) is called
3. ⚡ **error**: When reject(error) is called

🎯 State Transition Example

```
let anotherPromise = new Promise(function(resolve, reject) {  
  resolve("I am surely going to get resolved!");  
  
  reject(new Error("Will this be ignored?")); // 🚫 This is ignored  
  resolve("Ignored?"); // 🚫 This is also ignored  
});
```

💡 Knowledge Check:

- ▶ 🗝 Can a Promise change its state from fulfilled to rejected?

🎭 5. Handling Promises

🏰 The Three Handler Methods

🎉 1. .then() - Handling Success

```
promise.then(  
  (result) => {  
    console.log(result); // 🎉 Handle success  
  }  
);
```

🚨 2. .catch() - Handling Errors

```
promise.catch(  
  (error) => {
```

```
        console.error(error); // ✨ Handle error
    }
};
```

🔪 3. .finally() - Cleanup

```
promise.finally(
  () => {
    // 🎲 This runs regardless of success or failure
    console.log("Cleanup code here");
  }
);
```

🎮 Complete Example

```
let loading = false;
const promise = new Promise(function(resolve, reject) {
  loading = true; // 🔄 Start loading
  // 🌐 Simulate API call
  resolve("I am resolved...");
});

promise
  .then((result) => {
    console.log(result); // 🎉 Success!
  })
  .catch((error) => {
    console.error(error); // ✨ Error!
  })
  .finally(() => {
    loading = false; // 🎲 Stop loading
  });
```

💡 Knowledge Check:

- 🙋 What's the difference between .then() and .catch()?

🔗 6. Promise Chaining and Rules

🏆 The Five Golden Rules of Promise Chaining

📄 Rule 1: Handler Methods

- ✅ Every fulfilled promise provides a `.then()` handler

- ❌ Every rejected promise provides a `.catch()` handler

⚡ Rule 2: Three Things You Can Do in `.then()`

1. 🔄 Return another Promise (for async operations)
2. 💎 Return a simple value (for sync operations)
3. ✨ Throw an error

🎨 Examples of Rule 2

🔄 Returning Another Promise:

```
let getUser = new Promise(function(resolve, reject) {
  const user = {
    name: '👤 John Doe',
    email: '📧 jdoe@email.com',
    password: '🔒 jdoe.password',
    permissions: ['🗄️ db', '💻 dev']
  };
  resolve(user);
});

getUser
  .then(function(user) {
    console.log(`👉 Got user ${user.name}`);

    return new Promise(function(resolve, reject) {
      setTimeout(function(){
        resolve('🌆 Bangalore')
      }, 2000)
    })
  })
  .then((address) => {
    console.log(`📍 User address is ${address}`);
  });
```

💎 Returning a Simple Value:

```
getUser
  .then(function(user) {
    console.log(`👉 Got user ${user.name}`);
    return user.email; // 📧 Simple value
  })
  .then(function(email) {
    console.log(`📧 User email is ${email}`);
  });
```

✨ Throwing an Error:

```

getUser
  .then(function(user) {
    console.log(`👤 Got user ${user.name}`);

    if (!user.permissions.includes("hr")) {
      throw new Error("🚫 You are not allowed to access the HR module");
    }

    return user.email;
  })
  .then((email) => {
    console.log(`📧 User email is ${email}`);
  })
  .catch((error) => {
    console.error(error); // ⚡ Catch the error
  });

```

🔄 Rule 3: Rethrowing from .catch()

You can rethrow errors from `.catch()` to handle them later:

```

let promise401 = new Promise(function(resolve, reject) {
  reject(401)
});

promise401
  .catch((error) => {
    console.log(error);
    if (error === 401) {
      console.log("🔄 Rethrowing 401");
      throw error; // 🔄 Rethrow for next catch
    } else {
      // 🛠 Handle other errors
    }
  })
  .then((result) => {
    console.log(result);
  })
  .catch((error) => {
    console.log(`📧 handling ${error} here`);
  });

```

📧 Rule 4: .finally() Passes Through

Unlike `.then()` and `.catch()`, `.finally()` doesn't process the result—it passes it through:

```

let promiseFinally = new Promise(function(resolve, reject) {
  resolve('🌟 Testing Finally.');
```

```
});

promiseFinally
  .finally(function() {
    console.log("🔪 Running Finally!"); // Cleanup
  })
  .then(function(result) {
    console.log(result); // 📄 Still gets "Testing Finally."
  });
```

⚠️ Rule 5: Multiple .then() ≠ Chaining

Calling `.then()` multiple times on the same promise is NOT chaining:

```
// ❌ This is NOT chaining - each .then() gets the same original value
let promise = new Promise(function(resolve, reject) {
  resolve(10);
});

promise.then(function(value) {
  console.log(value + 1); // 📄 11
});

promise.then(function(value) {
  console.log(value + 2); // 📄 12 (not 13!)
});
```

💬 Knowledge Check:

- ▶ 🔗 What's the difference between promise chaining and calling `.then()` multiple times?

🏰 7. Handling Multiple Promises

🎲 Promise.all()

Waits for ALL promises to resolve. If any promise rejects, the entire operation fails.

```
const promise1 = getPromise(URL1);
const promise2 = getPromise(URL2);
const promise3 = getPromise(URL3);

Promise.all([promise1, promise2, promise3])
  .then((results) => {
    console.log(results); // 📄 Array of all results
  })
  .catch((error) => {
```

```
console.error(error); // ❌ If any promise fails
});
```

Promise.any()

Resolves with the first promise that fulfills. Rejects only if ALL promises reject.

```
Promise.any([promise1, promise2, promise3])
  .then((result) => {
    console.log(result); // 🏆 First successful result
  })
  .catch((error) => {
    console.error(error); // ❌ All promises failed
  });
```

Promise.allSettled()

Waits for all promises to settle (resolve or reject) and returns their results.

```
Promise.allSettled([promise1, promise2, promise3])
  .then((results) => {
    console.log(results); // 📋 Array of {status, value/reason}
  });
```

Promise.race()

Resolves or rejects with the first promise that settles.

```
Promise.race([promise1, promise2, promise3])
  .then((result) => {
    console.log(result); // 🏁 First to complete
  })
  .catch((error) => {
    console.error(error); // ❌ First to fail
  });
```

Knowledge Check:

- ▶  When would you use Promise.all() vs Promise.any()?

8. Promise Utility Methods

Promise.resolve()

Creates a resolved promise with a given value:

```
Promise.resolve("🌟 Hello World")
  .then(value => console.log(value)); // "🌟 Hello World"

// 🔄 Equivalent to:
let promise = new Promise(resolve => resolve("🌟 Hello World"));
```

❌ Promise.reject()

Creates a rejected promise with a given reason:

```
Promise.reject("🌟 Error occurred")
  .catch(error => console.log(error)); // "🌟 Error occurred"

// 🔄 Equivalent to:
let promise = new Promise((resolve, reject) => reject("🌟 Error occurred"));
```

🍕 9. Real-World Example: PizzaHub App

Here's a practical example showing how promises work in a real application:

```
// 🛒 API Functions
let getShopIds = () => {
  const url = `api/pizzahub`;
  return query(url);
};

let getPizzaList = (shopId) => {
  const url = `api/pizzahub/pizzas/${shopId}`;
  return query(url);
};

let getMyPizzaWithAddOn = (pizzas, type, name) => {
  let myPizza = pizzas.find((pizza) => {
    return pizza.type === type && pizza.name === name;
  });
  const url = `api/pizzahub/beverages/${myPizza.id}`;
  return query(url);
};

let performOrder = (result) => {
  return query(`api/order`, {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
    },
  });
};
```

```

    },
    body: JSON.stringify({
      pizzaId: result.pizzaId,
      beverageId: result.id,
    }),
  });
};

// 🔗 Promise Chain
function orderPizza(type, name) {
  getShopIds()
    .then((shopIds) => getPizzaList(shopIds[0]))
    .then((pizzas) => getMyPizzaWithAddOn(pizzas, type, name))
    .then((pizzaWithAddOns) => performOrder(pizzaWithAddOns[0]))
    .then((order) => confirmOrder(type, name, order.createdAt))
    .catch(function(error) {
      console.log(`😞 Bad luck, No Pizza for you today!`);
    });
}

// 🍕 Usage
orderPizza("🌿 veg", "🍕 Margherita");

```

🌟 10. Common Promise Patterns and Best Practices

🛡️ Error Handling Best Practices

1. ✅ Always include `.catch()` at the end of promise chains
2. 📝 Use specific error messages
3. 📊 Log errors appropriately
4. 🔄 Provide fallback values when possible

🔗 Promise Chain Best Practices

1. 📖 Keep chains readable with proper formatting
2. 🚫 Avoid deeply nested `.then()` calls
3. ➡️ Use arrow functions for simple operations
4. 🏠 Return promises or values explicitly

⚡ Performance Considerations

1. 🎯 Use `Promise.all()` for parallel operations
2. 🚫 Avoid creating unnecessary promises
3. ⚡ Handle errors early when possible

4.  Use `Promise.allSettled()` when you need all results
-

11. Interview Questions & Answers

Common Promise Interview Questions

Q1: What's the difference between callbacks and promises?

- ▶  Click to see answer

Q2: What happens if you don't handle promise rejection?

- ▶  Click to see answer

Q3: Can you convert a callback to a promise?

- ▶  Click to see answer

Q4: What's the difference between `Promise.all()` and `Promise.allSettled()`?

- ▶  Click to see answer
-

Practice Tasks

Task 1: Create Basic Promises

- ▶  Create a promise that resolves after 2 seconds with the message "Timer completed"

Task 2: Promise Chain

- ▶  Create a chain that multiplies a number by 2, then adds 10, then checks if result > 20

Task 3: Handle Multiple Promises

- ▶  Use `Promise.all()` to handle three API calls simultaneously

Task 4: Error Handling

- ▶  Create a promise that randomly resolves or rejects, handle both cases

Task 5: Fetch Data Chain

- ▶  Create a user authentication flow using promises

⚡ Task 6: Promise Race

- ▶ 🕒 Create a timeout mechanism using `Promise.race()`

📁 Task 7: Advanced Promise Chaining

- ▶ 🔄 Create a file processing pipeline using promises

📊 Task 8: Promise.allSettled() Usage

- ▶ 🎭 Handle mixed success/failure scenarios

🏆 Summary

JavaScript Promises are essential for modern asynchronous programming. Key takeaways:

1. 🎯 **Promises** represent future values and help avoid callback hell
2. 📊 **Three states:** pending, fulfilled, rejected
3. 🎭 **Handler methods:** `.then()`, `.catch()`, `.finally()`
4. 🔗 **Chaining rules** enable complex async flows
5. 🏰 **Multiple promise handling** with `Promise.all()`, `Promise.any()`, etc.
6. ✨ **Best practices** include proper error handling and readable chains

Master these concepts to write clean, maintainable asynchronous JavaScript code! 🚀 ✨

🌟 Keep Learning, Keep Growing! 🌟

"The future belongs to those who learn more skills and combine them in creative ways." 🧠

🚀 Let's Connect & Level Up Together

Follow me for daily JavaScript tips, insightful notes, and project-based learning.

🔗 [X \(Twitter\)](#)

💼 [LinkedIn](#)

💬 [Telegram](#)

📖 [Threads](#)

🔍 Dive into the full notes on GitHub → [40 Days JavaScript](#)

📅 *Next: Day - 24: Async Await* 📅

🚀 Happy JavaScript coding! 🚀